# Chapter 5
# Apache Portable Runtime (apr)

**Tóm tắt:** Trong chương này, chúng ta thảo luận về thư viện API khung phát triển ứng dụng Apache Portable Runtime (APR). Cụ thể, chúng ta thảo luận về vùng nhớ đệm (memory pool), tiến trình (process), luồng (thread) và nhóm luồng (thread pool) của APR. Chúng tôi trình bày các ví dụ sử dụng các hàm này trong môi trường thực tế. Các hàm thông tin tệp của APR được sử dụng kết hợp với IO ánh xạ bộ nhớ (memory mapped IO). Bảng băm APR được sử dụng để phát triển các ứng dụng đếm tần suất từ. Chúng ta sử dụng thư viện Memcache với APR, đồng thời cũng trình bày một ví dụ sử dụng bộ nhớ chia sẻ (shared memory) của APR.

**Nội dung**

5.1 | APR Memory Pool ... 114
5.2 | APR Processes ... 114
5.3 | APR Threads ... 116
5.4 | APR Thread Pool ... 117
5.5 | File information, IO, and Memory mapped files ... 119
5.6 | Hash tables ... 120
5.7 | Using Memcache with APR ... 122
5.8 | Shared memory with APR ... 124
5.9 | Conclusion ... 126

Apache Portable Runtime được viết để hỗ trợ tính di portability của máy chủ web HTTP Apache, và do đó cung cấp một lớp trừu tượng hệ điều hành (operating system abstraction). Hệ thống quản lý phiên bản Subversion (SVN) cũng sử dụng APR. Một khung phát triển ứng dụng khác là Nokia/Trolltech Qt Framework (xem Phần 19.1.3 để biết thêm chi tiết). APR có thể được sử dụng để cung cấp cho ứng dụng các chức năng cơ bản trong một API nhất quán trên nhiều nền tảng. Các chức năng chính của APR được hiển thị dưới đây:

1. Memory pool,
2. Atomic operations: giao diện với các thao tác nguyên tử (xem Phần 3.1.7.1),
3. Xử lý Đối tượng Động (Dynamic Object handling),
4. Các hàm môi trường,
5. Xử lý tín hiệu (Signal handling),
6. Thông tin tệp và IO,
7. Bảng băm,
8. Bản đồ bộ nhớ và cấp phát, bộ nhớ chia sẻ, thư viện mạng,
9. Thư viện luồng và tiến trình,
10. Biến Mutex và điều kiện, khóa đọc/ghi (read/writer locks).

Các tính năng này được mô tả chi tiết với sự trợ giúp của các ví dụ trong chương này.

## 5.1 APR Memory Pool

Những ưu điểm rõ rệt của việc sử dụng vùng nhớ đệm (memory pool) so với các hàm tiêu chuẩn malloc, new bao gồm:

1. Hiệu suất (Efficiency): một vùng nhớ đệm tùy chỉnh có thể hiệu quả hơn việc triển khai thư viện chuẩn,
2. Theo dõi tài nguyên (Resource tracking): vùng nhớ đệm có thể được sử dụng như một heap động nơi thời gian sống của bộ nhớ được theo dõi. Khi vùng nhớ được hủy, toàn bộ bộ nhớ sẽ tự động được thu hồi,
3. Constructor/Destructor: Các vùng nhớ đệm APR cung cấp các byte bộ nhớ thô, do đó chi phí của constructor không được cộng thêm (lưu ý, tính tiện lợi của việc khởi tạo cấu trúc dữ liệu phức tạp cũng không có).

Các hàm chính cho vùng nhớ đệm APR là:

```
apr_pool_create( apr_pool_t **pool, apr_pool_t *parent);
apr_allocator_t *apr_pool_allocator_get(apr_pool_t *pool);
apr_allocator_max_free_set( allocator, 64);
void *apr_palloc(apr_pool_t *pool, apr_size_t size);
apr_pool_clear (apr_pool_t *p);
apr_pool_destroy(apr_pool_t *p);
```

Sử dụng một pool cha, chúng ta có thể xây dựng phân cấp các pool, sao cho khi pool cha bị hủy, tất cả các pool con cũng bị hủy theo. Cũng có thể đăng ký một hàm callback sẽ được gọi khi một pool bị hủy. Các ví dụ sử dụng vùng nhớ đệm APR được trình bày trong phần tiếp theo.

## 5.2 APR Processes

Hãy xem danh sách chương trình được hiển thị trong Listing 5.1.

```
// \file calendar_proc.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of APR process type
#include <iostream>
#include <stdlib.h>
#include <assert.h>
#include <apr_general.h>
#include <apr_thread_proc.h>
static const char *PROGRAM_ARG[16];

int main( int argc, char *argv[ ] ) {
    apr_status_t retval;
    apr_pool_t *pool;
    apr_initialize();
    apr_pool_create( &pool, NULL );
    apr_procattr_t *attribute;
```

```
    /* first create the attribute */
    retval = apr_procattr_create( &attribute, pool );
    retval = apr_procattr_io_set( attribute, APR_NO_PIPE,
    APR_FULL_BLOCK, APR_NO_PIPE );
    //retval = apr_procattr_cmdtype_set( attribute, APR_PROGRAM_ENV );
    retval = apr_procattr_cmdtype_set( attribute, APR_PROGRAM_PATH );
    apr_proc_t calendar_process;
    PROGRAM_ARG[0] = "cal";
    PROGRAM_ARG[1] = "1";
    PROGRAM_ARG[2] = "2010";
    PROGRAM_ARG[3] = NULL;
    retval = apr_proc_create( &calendar_process, PROGRAM_ARG[0],
    PROGRAM_ARG, NULL, attribute, pool );
    if( retval != APR_SUCCESS ) exit( 1 ); //
    while( true ) { // read data from child process
        char buf[1024];
        retval = apr_file_gets( buf, sizeof( buf ), calendar_process.out );
        if( APR_STATUS_IS_EOF( retval ) ) break;
        std::cout << buf;
    }
    apr_file_close( calendar_process.out );
    int status;
    apr_exit_why_e why;
    retval = apr_proc_wait( &calendar_process, &status, &why, APR_WAIT );
    if( APR_STATUS_IS_CHILD_DONE( retval ) ) {
        std::cout << "WHY = ";
        switch( why ) {
            case APR_PROC_EXIT:
                { std::cout << "APR_PROC_EXIT"; break; }
            case APR_PROC_SIGNAL:
                { std::cout << "APR_PROC_SIGNAL"; break; }
            case APR_PROC_SIGNAL_CORE:
                { std::cout << "APR_PROC_SIGNAL_CORE"; break; }
        }
        std::cout << " Status = " << status;
    } else {
        std::cout << "still processing...";
    }
    std::cout << std::endl;
    apr_terminate();
    return (0);
}
```

Listing 5.1: Ví dụ về tiến trình APR chạy chương trình 'cal'

Trong danh sách trên, chúng ta thấy việc sử dụng một số hàm và cấu trúc dữ liệu của APR. Trước khi sử dụng chức năng của APR, APR phải được khởi tạo bằng cách sử dụng: apr_initialize(). APR có chức năng vùng nhớ đệm được sử dụng bởi hầu hết các hàm của APR cũng như các chương trình ứng dụng. Chúng ta tạo một vùng nhớ đệm bằng hàm apr_pool_create. Các dòng tiếp theo được sử dụng để tạo các thuộc tính tiến

trình kiểm soát hành vi của tiến trình, trạng thái tách rời (detach state) và IO. Trong trường hợp này, chúng ta muốn đọc từ đầu ra của tiến trình con, do đó chúng ta sử dụng APR_FULL_BLOCK cho đầu ra của nó.

Chúng ta chờ tiến trình kết thúc (nếu không chúng ta sẽ tạo ra một tiến trình thây ma - zombie process), và in mã thoát cũng như lý do thoát của nó.

### 5.3 APR Threads

Ngoài việc hỗ trợ đa nhiệm dựa trên tiến trình, APR cũng hỗ trợ đa luồng như được hiển thị trong Listing 5.2. Trong chương trình này, chúng ta tạo một luồng in ra một thông điệp. Hàm tạo luồng trong APR được hiển thị dưới đây:

```
/**
 * Create a new thread of execution
 * @param new_thread The newly created thread handle.
 * @param attr The threadattr to use to determine how to create the thread
 * @param func The function to start the new thread in
 * @param data Any data to be passed to the starting function
 * @param cont The pool to use
 */
apr_thread_create(apr_thread_t **new_thread,
                  apr_threadattr_t *attr,
                  apr_thread_start_t func,
                  void *data, apr_pool_t *cont);
```

```
// \file tapr.cpp
// \author Sandeep Koranne, (C) 2010
// \APR test and thread support

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <apr_thread_proc.h>
#include <apr_general.h>

#define NUM_THREADS 4

static void * APR_THREAD_FUNC comp(apr_thread_t *T, void *arg);

int main(int argc, char *argv[]) {
    apr_status_t retval;
    apr_pool_t *pool;
    apr_thread_t *THREADS[NUM_THREADS];
    apr_threadattr_t *thread_attribute;
    int i, j;

    apr_initialize();
    apr_pool_create(&pool, NULL);
```

```
    apr_threadattr_create(&thread_attribute, pool);
    for(i=0; i < NUM_THREADS; ++i) {
        retval = apr_thread_create(&THREADS[i], thread_attribute,
                                   comp, NULL, pool);
    }
    for(i=0; i < NUM_THREADS; ++i) {
        retval = apr_thread_join(&retval, THREADS[i]);
    }
    apr_terminate();
    return 0;
}

static void * APR_THREAD_FUNC comp(apr_thread_t *T, void *arg) {
    printf("\n Inside apr_thread function: comp");
    apr_thread_exit(T, APR_SUCCESS);
    return NULL;
}
```
Listing 5.2: Ví dụ về luồng APR

### 5.4 APR Thread Pool

Lên lịch các tác vụ sao cho CPU được sử dụng hiệu quả là một bài toán không tầm thường khi bài toán không đồng nhất và có thời gian chạy phụ thuộc vào dữ liệu. Trong tình huống như vậy, thường dễ dàng hơn khi thiết kế một nhóm luồng (thread pool) có thể được gán động để thực hiện tính toán. Vì mục đích này, APR có cơ sở hạ tầng nhóm luồng. Một ví dụ về nhóm luồng để thực hiện tính toán nặng được hiển thị trong Listing 5.3.

```
// \file thpool.cpp
// \author Sandeep Koranne, (C) 2010
// Fibonacci: (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89)
// Sum prefix-sum of F: (0, 1, 2, 4, 7, 12, 20, ...)

#include <stdio.h>
#include <pthread.h>
#include <string.h>
#include <stdlib.h>
#include <apr_general.h>
#include <apu.h>
#include <apr_reslist.h>
#include <apr_thread_pool.h>
#include <cassert>
#include <iostream>

struct Data {
    int s0, s1, N;
    unsigned int sum;
};
```

```
unsigned int CountCollatzReturn(unsigned int N) {
    unsigned int count = 0;
    while(1) {
        count++;
        if(N <= 1) return count;
        if(N % 2 == 0) N = N / 2;
        else N = 3 * N + 1;
    }
    return count;
}

#if 0
/**
 * function SumFibonacci
 * @param s0, s1 contain initial fibonacci numbers
 * @param N represents how many numbers to sum
 * @return sum of N fibo numbers from (s0,s1)
 * \description Algorithm is non-recursive
 */
unsigned int SumFibonacci(int s0, int s1, int N) {
    int i, j;
    unsigned int next = s0 + s1;
    unsigned int sum = 0;
    for(i=0; i < N-2; ++i) {
        next = s0 + s1;
        s0 = s1;
        s1 = next;
        sum += next;
    }
    return sum;
}
#else
unsigned int SumFibonacci(int s0, int s1, int N) {
    std::cout << "Running function " << __FUNCTION__ << std::endl;
    int i, j;
    unsigned int next = s0 + s1;
    unsigned int sum = 0;
    for(j=0; j < 100000000; ++j) {
        sum = 0;
        for(i=0; i < N-2; ++i) {
            next = s0 + s1;
            s0 = s1;
            s1 = next;
            sum += next;
        }
    }
    return sum;
}
#endif
```

```
void * APR_THREAD_FUNC FUNCTION(apr_thread_t *thd, void *arg) {
    struct Data *data = (struct Data *) arg;
    SumFibonacci(data->s0, data->s1, data->N);
    data->sum = CountCollatzReturn(data->sum);
    return APR_SUCCESS;
}

#define THREADED_VERSION
#ifdef THREADED_VERSION
int main(int argc, char *argv[]) {
    apr_status_t retval;
    apr_pool_t *pool;
    const int LOOP_COUNT = 8;
    const int Z = 7;
    int s0 = 0, s1 = 1;
    int i, j, rc = 0;
    unsigned int sum = 0;
    struct Data data[LOOP_COUNT];
    for(i=0; i < LOOP_COUNT; ++i) {
        data[i].s0 = i; data[i].s1 = i + 1; data[i].N = (i + Z);
    }
    apr_thread_pool_t *thrp;
    apr_initialize();
    apr_pool_create(&pool, NULL);
    retval = apr_thread_pool_create(&thrp, 2, 4, pool);
    if(retval != APR_SUCCESS) assert( 0 && "apr_thread_pool_create" );

    for(j=0; j < LOOP_COUNT; ++j) {
        retval = apr_thread_pool_push(thrp, FUNCTION, &data[j], 0, NULL);
        if(retval != APR_SUCCESS)
            assert( 0 && "apr_thread_pool_push" );
    }

    apr_size_t scount = apr_thread_pool_scheduled_tasks_count(thrp);
    std::cout << std::endl << scount << " tasks have been scheduled..." <<
std::endl;
    scount = apr_thread_pool_tasks_count(thrp);
    std::cout << std::endl << scount << " tasks are waiting..." << std::endl;
    scount = apr_thread_pool_tasks_run_count(thrp);
    std::cout << std::endl << scount << " tasks have completed..." << std::endl;

    while(scount != LOOP_COUNT) {
        scount = apr_thread_pool_tasks_run_count(thrp);
        if(scount == LOOP_COUNT) break;
        std::cout << std::endl << scount << " tasks have completed..." <<
std::endl;
        sleep(5);
    }
    // all work has been done
```

```
    retval = apr_thread_pool_destroy(thrp);
    for(i=0; i < LOOP_COUNT; ++i) {
        printf("\n sum (%d,%d,%d) = %d", i, (i+1), (i+Z), data[i].sum);
    }
    printf("\n");
    return (EXIT_SUCCESS);
}
#endif
```

Listing 5.3: Ví dụ về nhóm luồng APR

### 5.5 File information, IO, and Memory mapped files

APR hỗ trợ loại tệp trừu tượng apr_file_t cũng như loại thông tin tệp apr_finfo_t. Để mở một tệp để đọc, chúng ta có thể sử dụng hàm APR sau:

```
apr_status_t retval;
apr_pool_t *pool;
apr_pool_create(&pool, NULL);
apr_finfo_t file_info;
apr_file_t *fp;
retval = apr_file_open(&fp, fileName,
                       APR_READ | APR_BINARY,
                       APR_OS_DEFAULT, pool);
retval = apr_file_info_get(&file_info, APR_FINFO_SIZE, fp);
```

Hơn nữa, các tệp ánh xạ bộ nhớ (memory mapped files) có thể được tạo bằng hàm apr_mmap_create, ví dụ:

```
retval = apr_mmap_create(&mmap, fp, 0, file_info.size,
                         APR_MMAP_READ, pool);
// ... sử dụng dữ liệu mmap->mm dưới dạng const char *
apr_mmap_delete(mmap); // hủy ánh xạ tệp
```

### 5.6 Hash tables

Bảng băm trong APR được triển khai bằng cách sử dụng các pool APR. Các hàm sau đây hữu ích khi phát triển ứng dụng sử dụng bảng băm APR:

```
apr_hash_t *apr_hash_make(apr_pool_t *pool);
void *apr_hash_get(apr_hash_t *ht, const void *key, apr_ssize_t keylen);
```

Một ứng dụng đếm tần suất từ đơn giản sử dụng bảng băm APR được trình bày trong Listing 5.4.

```
// \file apr_hash_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Use of hash table in APR
#include <iostream>     // Program IO
#include <fstream>      // File IO
#include <cstdlib>      // exit
```

```
#include <string>       // STL string
#include <sstream>      // string iterator
#include <cassert>      // assertion checking
#include <cstring>      // memory functions
#include <apr_general.h>// APR basic functions
#include <apr_hash.h>   // APR hash tables

static int PrintHashFunction(void *dummy, const void *key,
                             apr_ssize_t klen, const void *val) {
    std::cout << (const char*)key << "\t" << (int)val << "\n";
    return 1;
}

static void CountWordFrequency(const char *filename) {
    std::ifstream ifs(filename);
    if(!ifs) {
        std::cerr << "Unable to open file: " << filename;
        exit(1);
    }
    apr_status_t retval;
    apr_pool_t *pool;
    apr_pool_create(&pool, NULL);
    apr_hash_t *word_hash = apr_hash_make(pool);
    assert(word_hash && "Word hash not created.");
    char line[1024];
    while(ifs) {
        ifs.getline(line, 1024);
        std::istringstream sstr(line);
        while(sstr) {
            std::string word; sstr >> word;
            if(word == "") break;
            std::cout << "Processing " << word << " " << word.size() << "\n";
            char *next_word = new char[word.size() + 1];
            strcpy(next_word, word.c_str());
            void *val = apr_hash_get(word_hash, (void*) next_word, word.size());
            if(val == NULL) { // lần đầu tiên
                apr_hash_set(word_hash, (void*) next_word, word.size(),
                             (const void*)(1));
            } else {
                int count = int(val) + 1;
                apr_hash_set(word_hash, (void*) next_word, word.size(),
                             (const void*)(count));
            }
        }
    }
    // bây giờ bảng băm đã được xây dựng.
    std::cout << "Hash has " << apr_hash_count(word_hash) << " elements.\n";
    apr_hash_index_t *hi;
    for(hi = apr_hash_first(pool, word_hash); hi; hi = apr_hash_next(hi)) {
        void *val;
```

```
        const void *key_word;
        apr_ssize_t key_len;
        apr_hash_this(hi, &key_word, &key_len, &val);
        std::cout << (const char*)(key_word) << "\t" << (int)val << "\n";
    }
    apr_pool_destroy(pool);
}

int main(int argc, char *argv[]) {
    if(argc != 2) {
        std::cerr << "Usage: ./apr_hash_example <file>..\n";
        exit(1);
    }
    apr_initialize();
    CountWordFrequency(argv[1]);
    std::cout << std::endl;
    apr_terminate();
    return (0);
}
```

Listing 5.4: Ví dụ sử dụng bảng băm APR để đếm tần suất từ

Trong Listing 5.4, cần lưu ý rằng khóa của bảng băm là địa chỉ của bộ nhớ đã được cấp phát chứa từ đó. Mặc dù hàm băm tính toán chính xác giá trị băm cho các từ khác nhau, việc sử dụng lại cùng một địa chỉ làm khóa cho các từ khác nhau sẽ không hoạt động, vì bảng băm không lưu trữ các khóa bên trong bảng băm (nó chỉ lưu trữ giá trị cho khóa đó, được chỉ mục bởi hàm băm đã tính toán).

Biên dịch chương trình trong Listing 5.4 và chạy nó trên một tập dữ liệu nhỏ cho chúng ta kết quả:

```
./apr_hash_example small_words.txt
Processing jill 4
Processing the 3
Processing jack 4
Processing jack 4
Processing jill 4
Hash has 3 elements.
the 1
jack 2
jill 2
```

Chương trình (trong Listing 5.4) cũng cho thấy hai cơ chế lặp bảng băm có sẵn trong APR: (i) sử dụng kiểu chỉ mục băm và (ii) sử dụng hàm callback.

## 5.7 Using Memcache with APR

Như chúng ta sẽ thảo luận trong Phần 9.6, memcached là một hệ thống bộ nhớ đệm đối tượng bộ nhớ phân tán hiệu suất cao. Nó được thiết kế để giảm tải cơ sở dữ liệu trong các ứng dụng web và tăng tốc độ tạo nội dung web động. Nó bao gồm một thành phần máy chủ quản lý thời gian sống của các đối tượng được lưu trong bộ nhớ cache có thể

được truy cập qua mạng. Sử dụng APR, chúng ta có thể tích hợp chức năng Memcache vào một ứng dụng. Tất nhiên, một tiến trình memcached đang chạy hợp lệ phải được thực thi trên máy chủ với cổng có thể truy cập được đối với chương trình khách. Ví dụ về việc sử dụng APR với Memcache được hiển thị trong Listing 5.5.

```
// \file aprcache.cpp
// \author Sandeep Koranne (C) 2010
// \description Use of memcache server with APR
#include <iostream>
#include <cassert>
#include <cstdlib>
#include <fstream>
#include <apr_general.h>
#include <apr_memcache.h>

int main(int argc, char *argv[]) {
    apr_initialize();
    apr_status_t retval;
    apr_pool_t *pool;
    apr_pool_create(&pool, NULL);

    // tạo một đối tượng máy chủ
    apr_memcache_server_t *myserver;
    retval = apr_memcache_server_create(pool, "localhost", 11211,
                                        1, 10, 10, 1000, &myserver);
    if(retval != 0) {
        std::cerr << "Unable to create server object..\n"; exit(1);
    }

    apr_memcache_t *mycache;
    retval = apr_memcache_create(pool, 2, 0x0, &mycache);
    if(retval != 0) exit(1);

    retval = apr_memcache_add_server(mycache, myserver);
    if(retval != 0) {
        std::cerr << "Unable to add server object..\n"; exit(1);
    }

    retval = apr_memcache_enable_server(mycache, myserver);
    if(retval != 0) {
        std::cerr << "Unable to enable server object..\n"; exit(1);
    }

    // kiểm tra xem bạn có thể truy xuất thông tin này hay không
    char *baton = NULL;
    apr_size_t baton_len = 0;
    retval = apr_memcache_getp(mycache, pool, "ABCDE",
                               &baton, &baton_len, 0x0);
```

```
    // Một ứng dụng sử dụng memcache phải được chuẩn bị
    // để giả định dữ liệu không có mặt trên máy chủ.
    if((retval != 0) || (baton == NULL) || (baton_len == 0)) {
        std::cout << "Data for key ABCDE does not exist in server.\n";
    } else {
        std::cout << "Memcache server returned : " << baton << std::endl;
    }

    apr_memcache_server_t *server =
        apr_memcache_find_server(mycache, "localhost", 11211);
    if(server) {
        std::cout << "Found memcached server running...\n";
    } else {
        std::cout << "Unable to find memcached server running...\n";
        exit(1);
    }

    // lấy thống kê máy chủ
    apr_memcache_stats_t *stats;
    retval = apr_memcache_stats(myserver, pool, &stats);
    if(retval != 0) {
        std::cerr << "Unable to collect memcache server stats..\n";
        exit(1);
    }

    std::cout << "\nMemcache server stats...." << std::endl
              << "\nVersion information : " << stats->version
              << "\nPID                 : " << stats->pid
              << "\nUptime              : " << stats->uptime
              << "\nCurrent items stored: " << stats->curr_items
              << "\nBytes used          : " << stats->bytes
              << "\nCurrent connections : " << stats->curr_connections
              << "\nBytes written       : " << stats->bytes_written;

    std::cout << std::endl;
    apr_terminate();
    return (0);
}
```

Listing 5.5: Sử dụng memcache với APR

Biên dịch và chạy chương trình này cho chúng ta kết quả:

```
./aprcache
Memcache server returned KEY
Found memcached server running...
Memcache server stats....
Version information : 1.4.5
PID                 : 30661
```

```
Uptime              : 1920
Current items stored: 1
Bytes used          : 57
Current connections : 10
Bytes written       : 9251

Data for key ABCDE does not exist in server.
Found memcached server running...
Memcache server stats....
Version information : 1.4.5
PID                 : 30661
Uptime              : 1991
Current items stored: 0
Bytes used          : 0
Current connections : 10
Bytes written       : 10050
```

Có thể thấy ở dòng 38 của Listing 5.5 rằng chúng ta cài đặt dữ liệu trên máy chủ với thời gian sống là 100 giây. Điều này có nghĩa là sau 100 giây, dữ liệu bị loại bỏ khỏi bộ nhớ cache. Bằng cách biên dịch chương trình có và không có phần chèn dữ liệu, chúng ta có thể thấy hiệu ứng loại bỏ bộ nhớ cache như trong đoạn bản ghi ở trên.

## 5.8 Shared memory with APR

Như chúng ta đã thấy ở trên, APR có các hàm để gọi các tiến trình con và đa xử lý. Giao tiếp giữa các tiến trình (inter-process communication) liên quan đến các phân đoạn dữ liệu chia sẻ lớn thường được thực hiện bằng cách sử dụng bộ nhớ chia sẻ (shared memory). Các phân đoạn bộ nhớ chia sẻ là các vị trí bộ nhớ có thể được truy cập bởi nhiều tiến trình. Tất cả các tiến trình phải đồng ý (hoặc được thông báo về) tên phân đoạn mà qua đó bộ nhớ chia sẻ sẽ được truy cập. Giống như bất kỳ tài nguyên được chia sẻ nào, các vùng tranh chấp (critical sections) cần được bảo vệ chống lại việc sử dụng vô tình bởi một tiến trình khác. Hàm APR apr_shm_create tạo ra một phân đoạn chia sẻ có tên và kích thước cho trước. Hàm apr_shm_destroy hủy phân đoạn chia sẻ. Trong thời gian phân đoạn chia sẻ hoạt động, địa chỉ bộ nhớ của nó có thể được tính toán bằng hàm apr_shm_baseaddr_get trả về một handle cho bộ nhớ cơ bản. Ví dụ về việc tạo phân đoạn bộ nhớ chia sẻ được hiển thị trong Listing 5.6.

```
// \file aprshmA.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of shared memory using APR
#include <iostream>
#include <cstdlib>
#include <apr_general.h>
#include <apr_shm.h>

static const char SHM_FILE_NAME[] = "SHM1234MHS"; // nên chung

int main(int argc, char *argv[]) {
    apr_initialize();
```

```
    apr_status_t retval;
    apr_pool_t *pool;
    apr_pool_create(&pool, NULL);
    apr_shm_t *shm_segment;
    retval = apr_shm_create(&shm_segment, 1024, SHM_FILE_NAME, pool);
    if(retval != 0) {
        std::cerr << "Unable to create shared memory segment..\n";
        exit(1);
    }
    int *data = (int *) apr_shm_baseaddr_get(shm_segment);
    for(int i=0; i < 10; ++i) data[i] = i;

    int x;
    std::cout << "Enter a number when done with programB:";
    std::cin >> x;
    std::cout << std::endl;
    apr_shm_destroy(shm_segment);
    apr_pool_destroy(pool);
    apr_terminate();
    return (0);
}
```

Listing 5.6: Ví dụ sử dụng bộ nhớ chia sẻ với APR

Tiến trình còn lại yêu cầu quyền truy cập vào phân đoạn chia sẻ bằng cách sử dụng hàm apr_shm_attach với cùng một tên tệp như tiến trình đã tạo phân đoạn chia sẻ. Một khi phân đoạn đã được đính kèm, địa chỉ cơ sở (baseaddress) và kích thước phân đoạn có thể được kiểm tra. Ví dụ về việc tạo phân đoạn bộ nhớ chia sẻ được hiển thị trong Listing 5.7.

```
// \file aprshmB.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of shared memory using APR
#include <iostream>
#include <apr_general.h>
#include <cstdlib>
#include <apr_shm.h>

static const char SHM_FILE_NAME[] = "SHM1234MHS"; // nên chung

int main(int argc, char *argv[]) {
    apr_initialize();
    apr_status_t retval;
    apr_pool_t *pool;
    apr_pool_create(&pool, NULL);
    apr_shm_t *shm_segment;
    retval = apr_shm_attach(&shm_segment, SHM_FILE_NAME, pool);
    if(retval != 0) {
        std::cerr << "Unable to attach shared memory segment..\n";
```

```
        exit(1);
    }
    apr_size_t data_size = apr_shm_size_get(shm_segment);
    std::cout << "size of shared memory segment = " << data_size << "\n";
    int *data = (int *) apr_shm_baseaddr_get(shm_segment);
    for(int i=0; i < 10; ++i) {
        if(data[i] != i) {
            std::cerr << "Data corrupted in shared segment..\n";
            exit(1);
        }
    }
    std::cout << std::endl << "Data in shared segment is valid.\n";
    apr_shm_detach(shm_segment);
    apr_pool_destroy(pool);
    apr_terminate();
    return (0);
}
```

Listing 5.7: Sử dụng phân đoạn bộ nhớ chia sẻ với APR

Thư viện Qt cũng có chức năng phân đoạn bộ nhớ chia sẻ có sẵn như một phần của API lập trình ứng dụng của nó, xem Phần 19.1.4.1 để biết thêm chi tiết. Đối với chức năng bộ nhớ chia sẻ POSIX, hãy chạy lệnh man shm_overview để có cái nhìn tổng quan về các hàm bộ nhớ chia sẻ trong POSIX.

## 5.9 Conclusion

Apache Portable Runtime (APR) quả thực là một thư viện phát triển ứng dụng linh hoạt. Nó không chỉ cung cấp một tập hợp các quy trình độc lập với hệ điều hành với ngữ nghĩa được xác định trước trên nhiều loại máy móc và tổ hợp hệ điều hành, mà còn cung cấp các tiện ích khung phát triển ứng dụng hữu ích như bảng băm, vùng nhớ đệm, hỗ trợ đa luồng và đa tiến trình, ánh xạ bộ nhớ và truy cập mạng.